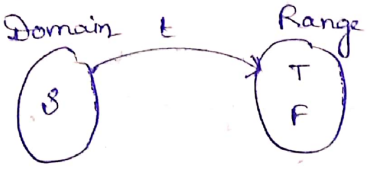
Date: 16/08/2023

Theoretical Computer Science:-

1) Methods of proof
2) Induction principle
3) Mathematical reasoning

} Study 1st book for example and beef book Ma'am's preference.

- When the range set is of truth values the function is called predicates.



$L(S) \in \{T, F\}$

- Mathematical Reasoning: A statement is either true or false.
- $p(x)$, predicate function that is true for some value of x it is either true or false.

  Say, p is tree then,

  For, x = Banyan $p(x)$: Banyan Tree = True

  x = Rohu $p(x)$: Rohu Tree = false.

- Is a predicate a relation?
- Fundamental binary relation: $\wedge, \vee, \sim, \rightarrow$
- Propositional logic statements denoted as P, Q, etc.
- Mostly, Hardly, Sometimes, etc. are not a part of reasoning. These prefixes don't work here.
- Correctness and incorrectness is not same as true or false.

$N = \{S\}$

$S \in N$

Grammar:-

- $G = (N, T, P, S)$, $S \in N$

  N = Set of non-terminals

  T = Set of terminals

  P = Set of rules (Production rule)

  S = Set of ↑ A special symbol (start symbol) (an element of N)

- For machine language, $T = \{0, 1\}$

* derivation must start with S.
'$\Rightarrow$' means derivation
'$\rightarrow$' means production/replacement.

- For C language terminals are : {, }, if, =, case,
- In any code, if we blank out all the variables, then what remains is a subset of the terminals.
- Language generated by a grammar is denoted by $L(G)$.
- For machine language {0, 1} is the only alphabet.
- $G = (N, T, P, S)$, $S \in N$ $\quad T = \Sigma = \{0, 1\}$ $\quad \Sigma_1 = \{a, b\}$
- L_1, the language containing strings containing odd no. of a's.

  Then, $G_1 = (N, T, P, S)$ $\quad S \in N$

  $N = \{S\}$

  $\Sigma / T = \{a\}$ = alphabet of the language

  *S: Start symbol

  $L_1 = \{a, aaa, aaaaa, \ldots\}$

  String/Word over an of alphabet.

  *2 P : 1. $S \rightarrow a$ (a production rule must have S at LHS)
  The RHS replaces the LHS
  ~~eg~~ *$S \underset{1}{\Rightarrow} a$
  2. $S \rightarrow asa$, $S \underset{2}{\Rightarrow} asa \underset{1}{\Rightarrow} aaa$ *'

  Σ is the symbol used to denote the alphabet used in a language.
- A Language L_1 over alphabet 'a' containing words of odd length.

  $\Sigma = \{a\}$, a is a symbol of the alphabet.
- Language is a set of words ~~genera~~ strings over an alphabet generated by a grammar. E.g. $L_1 = \{a, aaa, \ldots\}$
- For a language with even no. of a's,

  $L_2 = \{e, aa, \ldots\}$

  e = empty symbol, also ϵ can be written.
  = it has length zero. (It is not ϕ)

(1)

- $L_0 = \{\epsilon\}$, it is ~~not~~ also not ϕ. $|L_0| = 1$
- $|aaa| = 3$, $|a| = 1$, $|\epsilon| = 0$; it means 0 no. of a's are used for this symbol.

- $a \epsilon a \equiv aa$
  $\epsilon aa \equiv aa$
  $aa\epsilon \equiv aa$

$a^0 \Rightarrow \epsilon$ means zero occurance of a is ϵ.

3. $S \to \epsilon$
1. $S \to a$
2. $S \to aSa$

$w = aaaaa$

$S \Rightarrow aSa$
$\Rightarrow aaSaa$
$\Rightarrow aaaaa$
$\Rightarrow aaaaa$

- ~~*~~ 7 length word. G_1 cannot produce L_2.

$S \underset{2}{\Rightarrow} aSa$
$\underset{2}{\Rightarrow} aaSaa$
$\underset{2}{\Rightarrow} aaaaaaa$
$\underset{1}{\Rightarrow} aaaaaaa$

Sentential forms process is called deriving a word/producing a word.

$a \epsilon a$
aa

- Sentential forms
- Set of production rule cannot produce an invalid words

- ~~*~~2 P_1: 1. $S \to a$
  2. $S \to aAa$
  3. $A \to a$

$G_1' = (N, T, P_1, S)$
$N = \{S, A\}$

e.g 7 length word.

$S \underset{2}{\Rightarrow} aAa$
$\underset{2}{\Rightarrow} aaAaa$
$\underset{2}{\Rightarrow} aaaAaaa$
$\underset{3}{\Rightarrow} aaaaaaa$

$S \to \epsilon$
$|S| = 1$
$|\epsilon| = 0$

- RHS must be greater than or equal to LHS (exception ϵ production)
- ~~Set of non terminal o~~ $N \to NUT$
  Production ~~rules~~ rules of are set of non-terminals and Non-terminals union terminals.

- 1. $S \to \epsilon$; ϵ-production.
  2. $S \to aSa$
  For producing L_2.

$S \underset{1}{\Rightarrow} \epsilon$

$S \underset{2}{\Rightarrow} aSa$
$\underset{1}{\Rightarrow} aa$

- $L\{a,b\}$: String of a's followed by equal no. of b's.

$L\{a,b\} = \{\epsilon, ab, aabb, aaabbb, \ldots\}$ aaba

$G = \{N, T, P, S\}$ N = set of symbols other than the alphabet used to generate valid words/strings of Language called non-terminals.

P: 1. $S \to \epsilon$

2. $S \to aSb$

T = set of terminal symbols.

>

18/08/2023

1. Symbols in Hindi - अ आ क ख . . . 1 , ; : ∪ . — → c

2. Alphabet - अ आ . . . (set of symbols) क ख . . . $\Sigma = \{a, b\}$ $\Sigma' = \{③, ④\}$

3. $\Sigma_3 = \{aa, bb\}$ $w_1 = aa$ (Length is 1)

$\Sigma_1 = \{a\}$ $|aa| = 1$ (aa ∈

$\Sigma_2 = \{a, b\}$ $|aa\,bb\,aa| = 3$

4. A string/word is generated using the symbols of alphabet for 0 or more times.

5. The symbols using which Language is generated is called alphabet.

6. $L_{\Sigma_1} = \{a, aa, aaa, aaaa, \ldots\}$ = countably infinite as each element can be mapped to $\mathbb{Z}_+$.
(1 2 3)

= set of strings over alphabet Σ_1,

$L_{\Sigma_1} = \{\epsilon, aa, aaaa, \ldots\}$

— 100

$\epsilon \cdot \Delta$

→

7. Notation for empty string might be Δ as well.

8. Grammar, $G = (N, T, P, S)$

$A \to B$: B replaces A. (definition as well as semantics).

(3)

9. $A \in N^+$ $A \in (NUT)^+$ i.e, the smallest word/string that can come out of (NUT) is of length 1. $B \in (NUT)^+$ or $(NUT)^*$ according to language.

10. $S \to \epsilon$ Epsilon production. It is not common in many language

11. $\Sigma_1 = \{a, b\}$

$L'_{\Sigma_1} = \{\Lambda, ab, aabb, \ldots\}$

$\Sigma_1 = \{0, 1\}$

$\Sigma_1 = \{a, b\}$ $|A| \leq |B|$

P: 1) $S \to \epsilon$
2) $S \to ab$
3) $S \to aSb$

$S \overset{1}{\Rightarrow} \epsilon$

$S \overset{2}{\Rightarrow} ab$

$S \overset{3}{\Rightarrow} aSb$

$S \overset{3}{\Rightarrow} aaSbb \overset{1}{\Rightarrow} aabb$

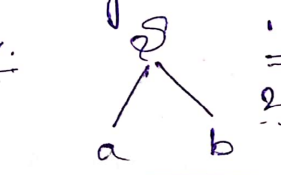
$w = aabb$

aaba

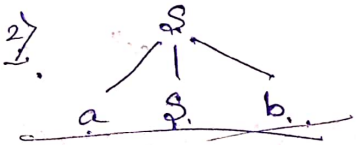
12. If Language does not contain empty word then we should avoid applying the epsilon rule.

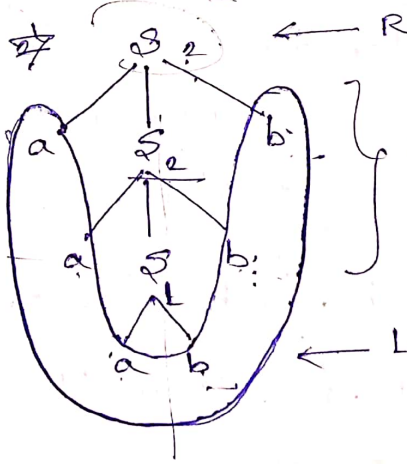
13. Other way to show derivation is: Derivation Tree.

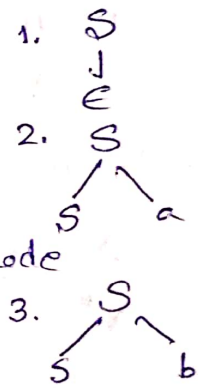
1) 1. $S \to ab$
2. $S \to aSb$



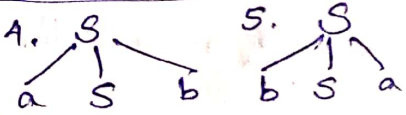
2)







$w = aaabbb$



14. $\Sigma_1 = \{a, b\} = T$ (a, b)

$L(\Sigma_1) = \{\epsilon, ab, ba, aaab, aba, baa, \ldots\}$

$G = (N, T, P, S)$ $N = \{S\}$ ("bbba")

P: 1) $S \to \epsilon$
2) $S \to Sa$
3) $S \to Sb$

~~2) $S \to aS$~~
~~3) $S \to Sb$~~

extra
4) $S \to aSb$
5) $S \to bSa$

$S \overset{5}{\Rightarrow} bSa$

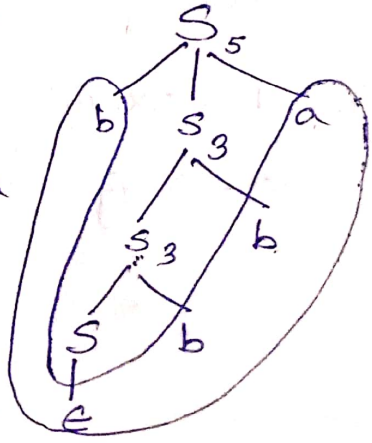
$\overset{3}{\Rightarrow} bSba$

$\overset{3}{\Rightarrow} bSbba$

$\overset{1}{\Rightarrow} bbba$

$b \in bba = bbba$

abaa



15) Find the production rule where the language is a palindrome.

$\Sigma_1 = \{a, b\}$

$L(\Sigma_1)$ = A language where words are palindrome

$= \{\epsilon, a, b, aba, bab, aa, bb, \ldots\}$

$G = (N, \Sigma_1, P, S)$ $\quad w = bab$

P : 1) $S \to \epsilon$

2) $S \to aS$

3) $S \to bS$

$N = \{S\}$

$\Sigma_1 = \{a, b\}$

$S \underset{3}{\Rightarrow} bS$

$\underset{2}{\Rightarrow} baS$

$\Rightarrow babS$

$\underset{1}{\overset{3}{\Rightarrow}} bab$

23/08/23

1. $G = (N, T, P, S)$

P : $A \to B$

$A \in N^+$

1. $S \to a$

2. $S \to aSa$

$\equiv S \to a \mid aSa$

(a or aSa)

2. ',' means concatenation. $\quad X^+ = \{X, X^2, X^3, \ldots\}$

$X = \{a, b\}$ → formalism (set of subsets)

$X^* = \{\phi_x, X, X^2, X^3, \ldots\}$ $\quad X^2$ is read as x^2 where 2 is superscript.

$X^2 = X \cdot X = \{a, b\} \cdot \{a, b\} = \{aa, ab, ba, bb\}$ $\quad X^0 = \epsilon$

$X^3 = X \cdot X^2 = \{a, b\} \cdot \{aa, ab, ba, bb\}$ $\quad \boxed{X^* = X^0 \cup X^+} \quad \boxed{= \epsilon \cup X^+}$

3. ϕ_x means the set with zero occurance of the elements of X.

$X^* = \{\underset{\epsilon}{\phi_x}, a, b, aa, ab, ba, bb, aaa, aab, aba\}$ $\quad \epsilon$

The superscript shows the length of the element.

4. $N = \{S, X, Y\}$

$\Sigma = \{0, 1\}$

④

5. $\Sigma = \{a, b, c\}$ — alphabet

L_Σ = All element of L_Σ is a language over Σ. the elements of L_Σ are strings over $\{a, b, c\}$ such that any no. of 'a's, followed by any no. of 'b's and followed by any no. 'c's.

~~$\{\phi\}$~~ $= \{\in, a, b, c, ab, ac, aa, bc, bb, cc, \ldots$
$aaa, aab, aac, abc, abb, \ldots, ccc, \ldots\}$

b c a

aaa bbb ccc
aab bbc
abb bcc
aac
acc
abc

* of length greater than or equal to 1.

$G = \{N, T, P, S\}$ $T = \{a, b, c\}$ $P: N^+ \rightarrow (NUT)^*$

$N = \{S, A, B, C\}$ $(NUT)^* = \{S, A, B, C, a, b, c\}$

Conventionally uses capital Letters.

* is only to accomodate $\in$ production.

P: 1. ~~$S \rightarrow \in$~~
1. $S \rightarrow ABC$
2. $A \rightarrow \in$
3. $B \rightarrow \in$
4. $C \rightarrow \in$
5. $A \rightarrow a$
6. $B \rightarrow b$
7. $C \rightarrow c$

$S \underset{1}{\Rightarrow} ABC \underset{5}{\Rightarrow} aBC \underset{3}{\Rightarrow} aC \underset{4}{\Rightarrow} a$

* as it is the only defaulter of $\|L\| \leq \|R\|$ rule.

But these production rules will create an $\in$ that is not part of the Language. So, we solve this another time.

- $L \in N^+$
  $R \in (NUT)^*$

$S \underset{2}{\Rightarrow} ABC \underset{3a}{\Rightarrow} aBC \underset{4}{\Rightarrow} aC \underset{8}{\Rightarrow} a$

~~4. $S \rightarrow \in$~~ we choose to remove it *

P: 1. $S \rightarrow ABC$
3a. $S \rightarrow a \mid \in$
4a. $B \rightarrow b \mid \in$
5. $S \rightarrow c \mid \in$
6a. ~~$A \rightarrow aA$~~ $A \rightarrow \in$
7. ~~$B \rightarrow Bb$~~ $B \rightarrow \in$
8. ~~$C \rightarrow cC$~~ $C \rightarrow \in$
9. $A \rightarrow aA$
10. $B \rightarrow Bb$
11. $C \rightarrow cC$

- Derive bbc

$S \underset{2}{\Rightarrow} ABC$

$\underset{6}{\Rightarrow} BC$

$\underset{10}{\Rightarrow} BbC$

$\underset{4}{\Rightarrow} bbC$

$\underset{5}{\Rightarrow} bbc$

- Derive aabbbcccc = W

$S \underset{2}{\Rightarrow} ABC$

$\underset{9}{\Rightarrow} aABC$

$\underset{9}{\Rightarrow} aaABC$

$\underset{6}{\Rightarrow} aaBC$

$\underset{10}{\Rightarrow} aaBbC$

$\underset{10}{\Rightarrow} aaBbbC$

$\underset{7}{\Rightarrow} aabbbC$

$\underset{11}{\Rightarrow} aabbbcC$

$\underset{11}{\Rightarrow} aabbbccC$

$\underset{81}{\Rightarrow} aabbbcccC$

$\underset{11}{\Rightarrow} aabbbccccC$

$\underset{8}{\Rightarrow} aabbbcccc$

Q. Positive no. of 'a's followed by positive no. of 'b's, followed by positive no. of 'c's.

$\Sigma = \{a, b, c\}$

$L_\Sigma = \{abc, aabc, abbc, abcc, \ldots\} = \{a^i b^j c^k \mid i, j, k \geq 1\}$

$G_1 = (N, T, P, S)$

$T = \{a, b, c\}$

$N = \{S\}$

P : 1. $S \to aA$
2. $A \to aA$
3. $A \to bB$
4. $B \to bB$
5. $B \to c$
6. $B \to cC$
7. $cC \to cc$ (flawed rule)

W = abc

$S \underset{1}{\Rightarrow} aA$

$\underset{3}{\Rightarrow} abB$

$\underset{5}{\Rightarrow} abc$

W = aabbbccc

$S \underset{1}{\Rightarrow} aA$

$\underset{2}{\Rightarrow} aaA$

$\underset{3}{\Rightarrow} aabB$

$\underset{4}{\Rightarrow} aabbB$

$\underset{4}{\Rightarrow} aabbbB$

$\underset{6}{\Rightarrow} aabbbcC$

$\underset{7}{\Rightarrow} aabbbcc$ ✗ wrong

(5)

Correct production rule:

P: 1. $S \rightarrow ABC$

2. $A \rightarrow a$

3. $A \rightarrow aA$

4. $B \rightarrow bB$

5. $B \rightarrow bB$

6. $C \rightarrow c$

7. $C \rightarrow cC$

25/08/2023

1. $\Sigma = \{a, b\}$ The language over $\Sigma = \{a, b\}$ of any arrangement of a and b's of any length.

$\{a, b\}^* \setminus \{a, b\}^+ = \{\epsilon\} \neq \emptyset$

$\{a, b\}^* = \{\epsilon, a, b, aa, ab, ba, bb, \ldots\}$ $(a+b)^* \approx (a \text{ or } b)^+ \approx (a \mid b)^+$

$\{a, b\}^+ = \{a, b, aa, ab, ba, bb, \ldots\}$

$L_0 = \{(a+b)^*\}$ L_Σ $A = \{a\}$ $A \cup B \Rightarrow (a+b)$

$B = \{b\}$ $(A \cup B)^2 \Rightarrow (a+b)^2 \Rightarrow \{a, b\}^2$

$L_1 = \{(a+b)^+\}$ strings of Length 2.

$= \{aa, ab, ba, bb\}$

$\Sigma^0 = \{\epsilon\}$ = a set with 0 occurance of a or b.

$\Sigma^1 = \{a, b\}$

$\Sigma^2 = \{aa, ab, ba, bb\}$

$\Sigma^3 = \{aaa, aab, aba, abb, baa, bba, bab, bbb\}$

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \ldots$$

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \ldots$$

2. $\Sigma = \{a, b\}$

$L_1 = \{(a+b)^+\}$

$G = (N, \Sigma, P, S)$

~~P: +~~

$N = \{S\}$

$T = \Sigma = \{a, b\}$

P: 1. $S \to a$

2. $S \to b$

3. $S \to SS$

aabbaa

bbaaba

$L_1 = \{(a+b)^+\} = \{a, b, aa, ab, ba, bb, \ldots\}$

Left most derivation = replace left most non-terminal with appropriate rule.

$W_1 = aabbaa$

$S \underset{3}{\Rightarrow} SS$

$\underset{3}{\Rightarrow} SSSS$

$\underset{3}{\Rightarrow} SSSSSS\cancel{SS}$

$\underset{3}{\Rightarrow} SSSSSS$

$\underset{1}{\Rightarrow} aSSSSS$

$\underset{1}{\Rightarrow} aaSSSS$

$\underset{2}{\Rightarrow} aabSSS$

$\underset{2}{\Rightarrow} aabbSS$

$\underset{1}{\Rightarrow} aabbaS$

$\underset{1}{\Rightarrow} aabbaa$

$W_2 = aab$

$S \underset{3}{\Rightarrow} SS$

$\underset{1}{\Rightarrow} aS$

$\underset{3}{\Rightarrow} aSS$

$\underset{1}{\Rightarrow} aaS$

$\underset{2}{\Rightarrow} aab.$

Q. The string over $\{a, b\}$ which should essentially begin with b.

$\Sigma = \{a, b\}$

$L = \{b, ba, bb, baa, bab, bbb, bba, \ldots\} = b(a+b)^*$

$G = (N, \Sigma, P, S)$

P: 1. $S \to b$

2. $S \to a$

3. $S \to \cancel{bS} SS$

$W = bbaba$

$S \underset{3}{\Rightarrow} bS$

$\underset{3}{\Rightarrow} bbS$

8. $\Sigma = \{a, b\}$ $\quad W = aabbbbaabb$

$L_3 = (aa + bb)^+$

23/11/22

→ Even length

→ Not a single occurance of a or b.

* Write the language of string even length, equal no. of a's and b's in any order, cannot have more than 2 successive a's and b's, substrings of a and b are not of odd length.

$L_4 = \{ab, ba, aabb, abab, baba, bbaa, abba, baab, \ldots\}$

$\neq$ ~~$(a^+ + b^+)^+$~~

$= (ab + ba)^+$

$a^+ = \{a, aa, aaa, \ldots\}$

$b^+ = \{b, bb, bbb, \ldots\}$

$a^+ + b^+ = ab, ba, abb \times \ldots$

$(aabb, bbaa)(ab + ba)^+ = \{ab, ba, abab, abba, baba, baab, \ldots\}$

$(aa + bb)^+ = \{aa, bb, aaaa, bbbb, aabb, bbaa, \ldots\}$

$G = (N, T, P, S)$

$N = \{S,$

$T = \{a, b\}$

P: 1. $S \to ab$

2. $S \to ba$

3. $S \to aSb$

4. $S \to bSa$

(NUT)

W = abbbaa

$S \xrightarrow[3]{} aSb$

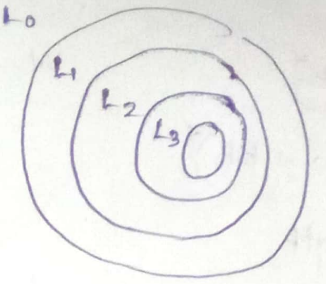
$\Rightarrow_3 abS$.

A

$aS \to$ —

NUT

P : $(NUT)^+ \rightarrow (NUT)^*$



Type-0 = Unrestricted grammar

P: $(NUT)^+ \rightarrow (NUT)^*$

Unrestricted $|L| , |R|$

Type-1 = Context sensitive Grammar

P: $(NUT)^+ \rightarrow (NUT)^*$

$|L| \leq |R|$ as well as $|L| \geq 1$

if $|L| = 1$, then $L \in N$

if $|L| > 1$, then $L \in (NUT)^+$

Type-2 = Context-free grammar. [LHS is going to be length 1]

P: ~~NUT)~~ $N \rightarrow (NUT)^*$ and ϵ production.

Type-3 = Regular Grammar (Left Linear Grammar or Right linear Grammar) No ϵ production.

P: $N \rightarrow (NUT)^+$ in this case RHS is mostly of length 2 (not more)

$R \in (NUT)^+$

$|L| \leq |R|$

eg. $S \rightarrow aS$ (RL)

or, $S \rightarrow Sa$ (LL)

consistently for all other production rules in each case.

It has to have a terminal in RHS.

a S	N	Sa
a A	2	Aa
b A		

30/08/2023

Type-0:

1. • $L_0 = \{L_1, \dots\}$ = unrestricted language.
   • Type-1: $L_1 \subset L_0$
   • Type-2: $L_2 \subset L_1 \subset L_0$
   • Type-3: $L_3 \subset L_2 \subset L_1 \subset L_0$

* The programming Languages are not ambiguous. This is what ~~di~~ makes a programming language ~~has~~ different ~~that~~ from a natural language.

2. $\Sigma = \{a, 1\}$
   $\Sigma' = \{a, b, c\}$
   $\Sigma_1 = \{x, z, \$, A\}$
   $\Sigma_1' = \{\epsilon\}$
   $\Sigma_1' \neq \phi$

   $||x|| = 1$ but $||\epsilon|| = 0$
   $|\Sigma_1| = 4$

3. $(a+b)$, $(a.b)$ here '+' and '.' are operators to denote regular expressions.

4. $(a+b)$: one occurance of a OR (+) one occurance of b
   $(a.b)$: a and b are in juxtaposition.

5. $R_1 = (a+b)^* = \{(a+b)^0, (a+b)^0 \cdot (a+b)^1, (a+b)^0, (a+b)^1 \cdot (a+b)^2, \dots\}$
   $= \{\epsilon, a, b, aa, ab, ba, bb, \dots\}$
   $= \Sigma^*$ where $\Sigma = \{a, b\}$
   ~~R_1 = Regular set~~ L'_Σ = Regular set
   $(a+b)^*$ = Regular expression G' = Regular Grammar.

   • Regular set
   Regular expression
   Regular Language.

6. $R_2 = a(a+b)^*$
   $= a\{\epsilon, a, b, aa, ab, ba, bb, \dots\}$
   $= a, aa, ab, aaa, aab, aba, abb, \dots\}$
   $R_3 = a(a+b)^*bb$
   $= a\{\epsilon, a, b, aa, ab, ba, bb, \dots\} bb = \{abb, aabb, abbb, \dots\}$

7. $L(R_2) = \{a, aa, \cancel{aab}, ab, aaa, aab, aba, abb, \ldots\}$

$G = (N, \Sigma, P, S)$

$N = \{S, A\}$

$\Sigma = \{a, b\}$

P: 1. $S \to a$ | $S \to aB$

2. $S \to aA$ | $A \to BA$

3. $A \to a$ | $B \to AB$

4. $A \to b$ | $B \to b$

5. $A \to aA$ | $B \to aB$

6. $A \to bA$ | $B \to bA$

~~7. $A \to \epsilon$~~ (not required).

Deriving an expression may be denoted as:

$S \overset{+}{\Rightarrow} w$

$S \underset{1}{\Rightarrow} a$, $S \underset{3}{\Rightarrow} \ldots$, etc.

sentential forms -

- SF which does not have any non-terminal symbol is called a string.

8. $L(R_3) = a(a+b)^* bb = \{abb, aabb, abbb, aabbb, ababb, \ldots\}$

$= a\{\epsilon, a, b, aa, ab, ba, bb, \ldots\} bb$

$= \{abb, aabb, abbb, aaabb, \ldots\}$

$G = (N, T, P, S)$

$N = \{S, A, B\}$

$T = \Sigma = \{a, b\}$

P: 1. $S \to \cancel{abb}\ AB$

2. $S \to abb$

3. $\cancel{S} \to \cancel{aAabb}\ a\cancel{S}bb$

4. $A \to a\cancel{A}$

5. $B \to b\cancel{B}$

6. $S \to aBbb$

7. $S \to BA$

$w = ababb$

$S \underset{3}{\Rightarrow} aSbb$

$\underset{7}{\Rightarrow} a\cancel{BA}bb$

$\cancel{\Rightarrow abBbb}$

$\underset{5}{\Rightarrow} abAbb$

$\underset{4}{\Rightarrow} ababb$

$w = ababb$

$S \underset{2}{\Rightarrow} aBAbb$

$\underset{8}{\Rightarrow} abAbb$

$\underset{6}{\Rightarrow} ababb$

$w = \cancel{aababbbb}$

$S \underset{1}{\Rightarrow} aABbb$

$\underset{9}{\Rightarrow} aaBBbb$

$\underset{7}{\Rightarrow} aabBBbb$

$\underset{8}{\Rightarrow} aabbBbb$

$\underset{10}{\Rightarrow} aabbbAbb$

$\underset{6}{\Rightarrow} aabbbabb$

or P: 1. $S \to aABbb$

2. $S \to aBAbb$

3. $S \to aAAbb$

4. $S \to aBBbb$

5. $A \to aA$

6. $A \to a$

7. $B \to bB$

8. $B \to b$.

9. $A \to aB$

10. $B \to bA$

Finite State Machines:-

Man, goat, cabbage, Fox problem.

M, G, C, F | ⊓→ Boat | ∅

1. State is a state of problem solving process.
2. Problem Input — Result Output
3. Initial state : (P, ∅)

   ⋮

   Final state : (∅, sol)

Algorithms:

Algorithms are ~~set of sequence~~ of ~~a~~ set of proper sequence of instructions where there might be none or n inputs that give an output and terminate infinite times.

Q. What is the difference between computation and calculation?

4. Every statement might have multiple instructions and each instruction might have multiple states.
5. Finite state Machines, $M = (Q, \Sigma, \delta, q_0, F)$

   Q = set of states $\quad q_0 \in Q, \quad F \subseteq Q$.

   Σ = Alphabet over which the value terms are defined.

   δ = Set of transition rules, $\delta : (Q \times \Sigma) \longrightarrow Q$

   q_0 = initial state

   F = ~~final state~~ set of final states.

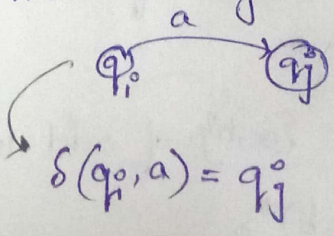
δ is a function from a set of pairs to Q.

$Q = \{q_0, q_1, q_e, q_f\}$

$F = \{q_1, q_f\}$

Graphical representation of δ:-

Transition diagram or graph.



$q_i, q_j \in Q$
$a \in \Sigma$

$\delta(q_i, a) = q_j$

31/08/2023

1. Δ → empty word

2. R: Class of regular languages

i. $\phi \in R$

ii. $\{\Delta\} \in R$

iii. $\{a\} \in R$

iv. L_1 and L_2, $L_1 \cup L_2$ will be regular set.

$r_1 \in L_1 \quad r_2 \in L_2 \Rightarrow L_1 \cup L_2$ as $r_1 + r_2$

v. L_1 and L_2, $L_1 . L_2$ as $r_1 r_2$, $L_2 . L_1$ as $r_2 r_1$

~~vi. Clean closure~~

vi. Kleene closure (* closure)

L', L'^*

$r \in L'$, r

* Regular language is synonymously used as regular set.

3. $\Sigma = \{a\}$

$L'_\Sigma = \{\Delta, a, aa, aaa, \ldots\} = a^*$ (Regular expression)

$w_6 = aaaaaa$

4. $\Sigma = \{a, b\}$

~~ab ba~~

$a^* + b^* = \{\Delta, a, aa, aaa, \ldots\}$ or $\{\Delta, b, bb, bbb, \ldots\}$

$= \{\Delta, a, b, aa, ab, ba, bb, aaa, abb, aab, bbb, \ldots\}$

$a^* b^* = \{\Delta, a, b, ab, \ldots\}$ a ~~ba~~

$\subseteq$ $(ab)^* = \{\Delta, ab, aabb, abab, ababab, \ldots\}$ ab. ab

$(a+b)^* = \{\Delta, a, b, ab, ba, bb, aa, \ldots\}$

a b .

Language over $\{a, b\}$ of even length.

- $(aa+bb+ba+ab)^*$

Language over $\{a, b\}$ of odd length ending with a.

- $(aa+bb+ba+ab)^* . a$

⑨

Language over {a, b} of odd length.

- $[\{(a+b)(aa+bb+ab+ba)^*\}+\{(aa+bb+ab+ba)^*(a+b)\}]$

5. $m = (Q, \Sigma, \delta, q_0, F)$

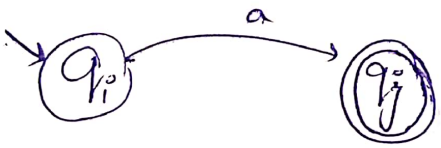
$F \subseteq Q$, $q_0 \in Q$

$\delta : (Q \times \Sigma) \longrightarrow Q$

$Q = \{q_0, q_i, q_j, q_f\}$

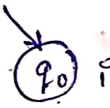
$\Sigma = \{a, b\}$

$\delta(q_i, a) = q_j$

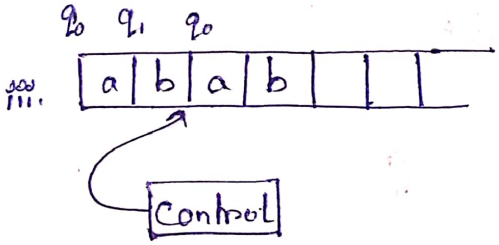


Transition graph is a labelled directed graph. $[G(V, E)]$

6. i. ϕ : $V = \phi$, $E = \phi$ $\quad G = (\phi, \phi)$

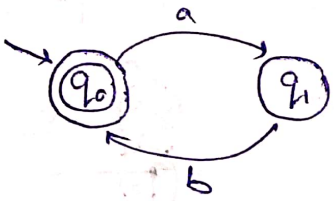
ii. $\in$: $V = \{q_0\}$, $\longrightarrow$  final state  initial state

iii.



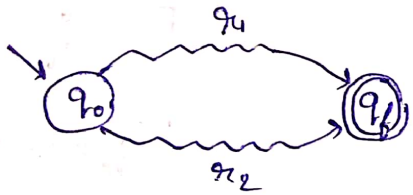
$\delta(q_0, a) = q_1$

$\delta(q_1, b) = q_0$



7. iv. $L_1 \cup L_2$

$r_1 + r_2$



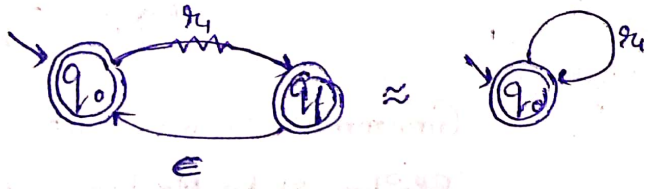
8. v. $L_1 . L_2$

$r_1 r_2$

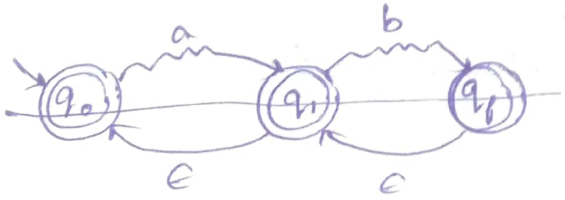
there might be an entire sub-graph.

9. vi L_1^*

r_1^*



10. $a^* b^* = \{\epsilon, a, b, ab, abb, aa, aaa, aa\,bbbb, \ldots\}$ 3



$m = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1$

$F = \{q_0, q_1, q_2\}$

$\delta(q_0, \epsilon) = q_0$

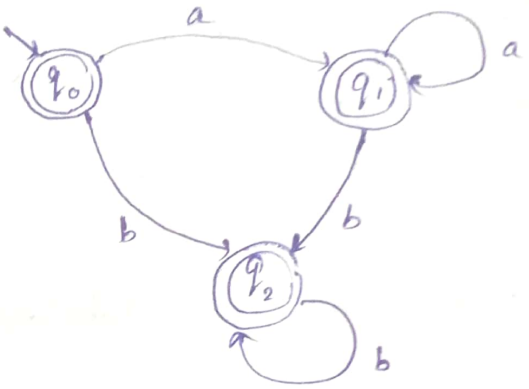
$\delta(q_0, a) = q_1$

$\delta(q_0, b) = q_2$

$\delta(q_1, b) = q_2$

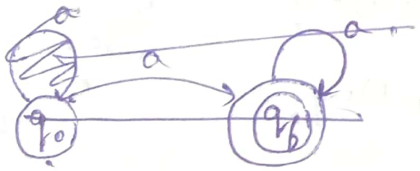
$\delta(q_1, a) = q_1$

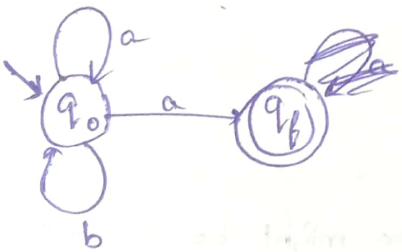
$\delta(q_2, b) = q_2$



11. $(a+b)^* a = \{\Delta, a, b, ab, ba, aa, bb, \ldots\}\, a$

$= \{a, aa, ba, aba, baa, aaa, bba, \ldots\}$





$Q = \{q_0, q_f$

$F = \{q_f$

$\delta(q_0, a) = q_f$

~~$\delta(q_f, a) = q_f$~~

$\delta(q_0, a) = q_0$

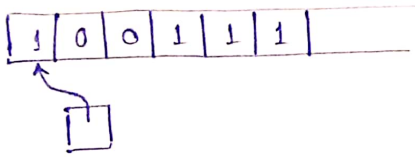
$\delta(q_0, b) = q_0$

12. $(a+b)^*$

Grammar = Generator

Finite State Machine = Recogniser.

01/09/2023



$M = \{q_0\}\ \{0,1\}\ \delta, q_0, \{q_0\}$

$\delta(q_0, \epsilon) = q_0$

$\delta(q_0, 1) = q_0$

$\delta(q_0, 0) = q_0$

Configuration:-

$Q \times \Sigma^* = \{\quad , w, \quad\}$

$Q \times \Sigma^* = \{\quad , 1001, \quad\}$

$(q, 1001)$

$Q \times \Sigma^*$ → denotes a configuration

(q, w) → part of configuration

- With a initial configuration, if the machine arrives to a configuration and the state is final state, then it is called <u>final configuration</u>.
- If not in final state → not accepting the string.

$(q_0, 1001) \vdash$

$\vdash (q', \epsilon) \quad \because q' \in F$

$\vdash (q_f, w') \quad q_f \in F$ (Indicates that string not accepted by machine)

$\vdash (q_i, \epsilon) \quad q_i \notin F$

$(q_0, 1001) \xrightarrow{(ii)} (q_0, 001)$

$\xrightarrow{(iii)} (q_0, 01)$

$\xrightarrow{(iii)} (q_0, 1)$

$\xrightarrow{(ii)} (q_0, \epsilon)$

↓ initial configuration

i) $\delta(q_0, \epsilon) = q_0$

ii) $\delta(q_0, 1) = q_0$

iii) $\delta(q_0, 0) = q_0$

→ For a given automata (Automata is a recogniser of the language),

$M_1 = \{q_0\}, \{0,1\}, \delta, q_0, \{q_0\}$

For any $w = 100111$, if from initial configuration we reach final configuration → Then accepted.

→ $w = aw'$ (Any arbitrary string)

$(q, w) \vdash (q_1, w')$ ⇒ Transition like this is only possible if

$\exists\ \delta(q, a) = q_1$

→

δ	0	1
q_0	q_0	q_0

← Another type representation of transition function.

→ $L = (1+0)(00+11)^+ 1$ $\qquad r_1 = (1+0)^*$

$r_2 = (1+0)(00+11)^+ 1$

$L_{r_2} = \{1$

$w = 1001$ (strings may begin with 0 or 1 but ends with 1)

→ ~~If~~ $(1+0)(00+11)^* 1$

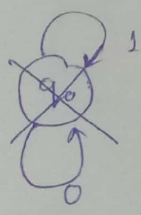
$w = \{1001$, 1111, 0001, 0111

for 0	for 1
0	100001
0	11111
0	10011
0	11001

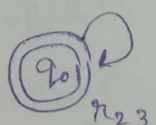
= total 12

→ $\Sigma = \{0, 1\}$

$r_2 = (1+0)^{r_{21}} (00+11)^{+\ r_{22}*} 1^{r_{23}}$

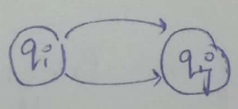


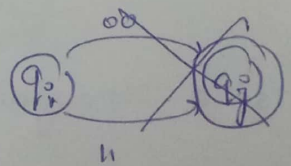
q_0



r_{21}

$0^* + 1^*$

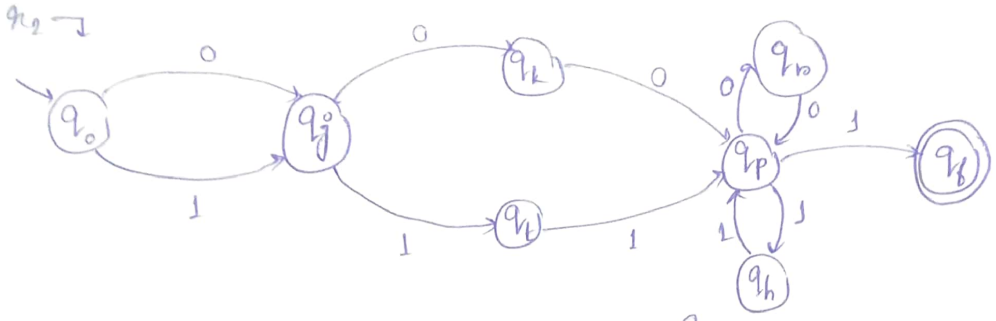




11

1000 000 1111

(11)



$Q = \{q_0, q_j, q_k, q_l, q_p, q_r, q_h, q_f\}$

δ	0	1
q_0	$\{q_j\}$	$\{q_j\}$
q_j	$\{q_k\}$	$\{q_l\}$
q_k	$\{q_p\}$	$\underline{\phi}$
q_l	$\underline{\phi}$	$\{q_p\}$
q_p	$\{q_r\}$	$\{q_f, q_h\}$
q_r	$\{q_p\}$	$\underline{\phi}$
q_h	$\underline{\phi}$	$\{q_p\}$
q_f	$\underline{\phi}$	$\underline{\phi}$

$(q_p, 11) \vdash (q_h, 1)$
(or)
$\vdash (q_f, 1)$
} non deterministic

→ Non-deterministic Automata → we get more that one state
$m = (Q, \Sigma, \delta, q_0, f) \quad \delta : (Q \times \Sigma) \to \mathcal{P}(\Phi)$

→ In above transition table we get non-deterministic automata, so no transition state can be replaced with "$\underline{\phi}$".

→ $(q_p, 11) \vdash (q_h, 11)$
$\vdash (q_p, 1)$
$\vdash (q_h, \epsilon)$ → not final state so another possibility will be,
$\vdash (q_p, 1)$
$\vdash (q_f, \epsilon)$

1. Page - 44 — Alphabets, strings, languages. — Ullman book
2. Formal languages and automata book.

$a^n b^{3n}$

$a^{n+3} b^n$

$n \geq 1$

$a^1 b^1 = aaaab$

$a^5 b^2 = aaaaabb$

$S \rightarrow aAB$

$B \rightarrow bB$

$B \rightarrow b$

$\rightarrow a \quad A \rightarrow \epsilon$

aaa a^{**}

aaa A B

II a) $L = \{a, ab, abb, abbb, \ldots\}$

1. $S \rightarrow a$
2. $S \rightarrow aB$
3. $B \rightarrow bB$
4. $B \rightarrow b$

$S \rightarrow aB$

$B \rightarrow bB$

$B \rightarrow \epsilon$

1. $S \rightarrow 0$
2. $S \rightarrow 1$
3. $S \rightarrow 0S0$
4. $S \rightarrow 1S1$

$L_4 = \{a^n b^{n-3} : n \geq 3\}$

$= \{a^3 b^0, a^4 b^1, a^5 b^2, \ldots\}$

$= \{aaa, aaaab, aaaaabb, \ldots\}$

Shreya.

1. $S \rightarrow aaaAB$
2. $A \rightarrow \epsilon$
3. $B \rightarrow \epsilon$
1. $B \rightarrow bB$
5. $A \rightarrow aA$

$S \xrightarrow[1]{} aaaAB$

$\xrightarrow[5]{} aaaaAB$

$\xrightarrow[5]{} aaaaaAB$

$\xrightarrow[2]{} aaaaaB$

$\xrightarrow[1]{} aaaaabB$

$\xrightarrow[1]{} aaaaabbB$

$\xrightarrow[3]{} aaaaabb$

13/09/2023

Non-Deterministic Finite Automata:-

① DFA: $\delta : Q \times \Sigma \rightarrow Q$ $\quad P(Q)$ = power set of Q

NFA: $\delta : Q \times \Sigma \rightarrow P(Q)$

δ	0	1
q_0		
q_1	$\{q_0\}$	
q_2		
⋮	⋮	⋮
q_f		

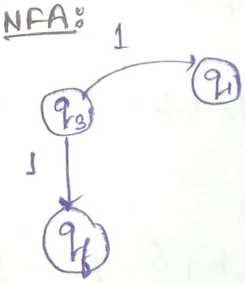
Transition table of NFA

δ	0	1
q_0		
q_1	q_0	
q_2		
⋮		
q_f		

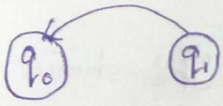
Transition Table of DFA

② NFA:

NFA:



DFA:



$\delta(q_3, 1) = q_1$

$\delta(q_3, 1) = q_f$

③ $G(V, E) \neq m = (Q, \Sigma, \delta, q_0, F)$

④ $(w, q_0) \vdash (w', q')$ • Configuration where transition function moves from q_0 to q'.

$\vdash \ldots$

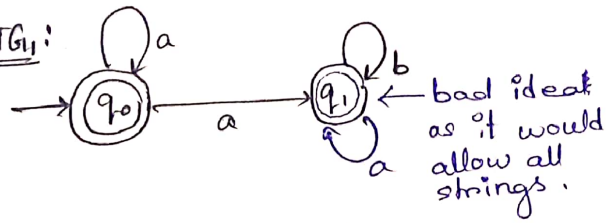
$\vdots$

$\vdash (\epsilon, q_f)$ • $\|w\| \geq 1$

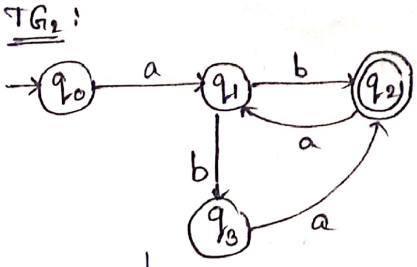
$\|\epsilon\| = 0$

⑤ Can NFA be a recogniser set for regular set or a language represented by a regular expression?

⑥. TG$_1$:



TG$_2$:



↳ This machine will hang if it enters a 'b'. At some point it will reach (ϵ, q_1) as q_1 in not a final state but won't terminate.

↳ It is a finite state machine.

↳ The strings:

$L = \{\epsilon, a, aa, aaa, \ldots\}$

↳ $(aaaa, q_0) \vdash (aaa, q_1)$ Not final state

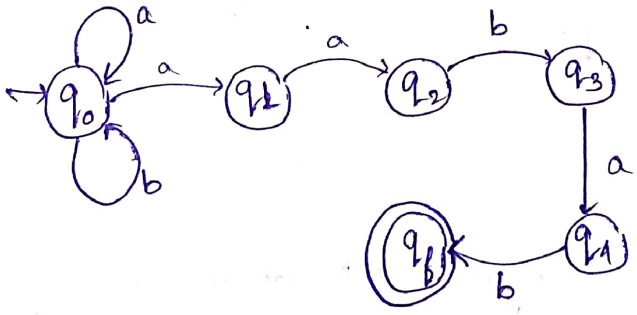
$\vdots$

$\vdash (\epsilon, q_1)$ Not final state either

This shows not to take invalid strings.

↳ $m_1 = (\{q_0, q_1\}, \{a, b\}, \delta, q_0, \{q_0\})$

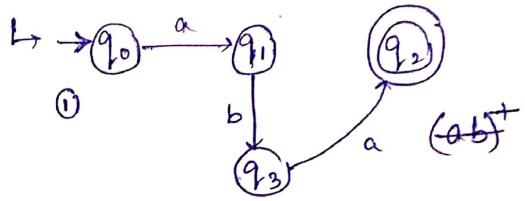
⑦. $(a+b)^* aabab$



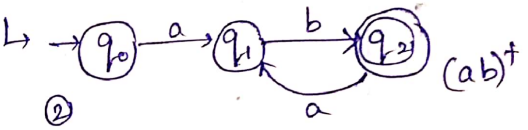
⑧. Automata + Transition Graph + Regular Expression.

↳ $m_2 = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, q_0, \{q_2\})$

↳



↳



↳ ② The smallest string accepted by this automata is 'ab'.

$L_{\{a,b\}} = \{ab, abab, ababab, \ldots, aba, abaaab, abaaba, \ldots\}$

↳ A string of length 7:

$w = abaabab$

$w' = aba, aba, aba$

$L_{\{a,b\}} = (ab \cup aab \cup aba)^*$

$= (ab + aab + aba)^*$

This is some other language

(9). **Theorem:** The class of Languages accepted by finite state machines is closed under union, concatenation, Kleene closure (star), complementation and intersection.

If $L_{FA} = \{L_1, L_2, \ldots, L_k, \ldots\}$

Then, $L_i \cup L_j \in L_{FA}$

$L_i . L_j \in L_{FA}$

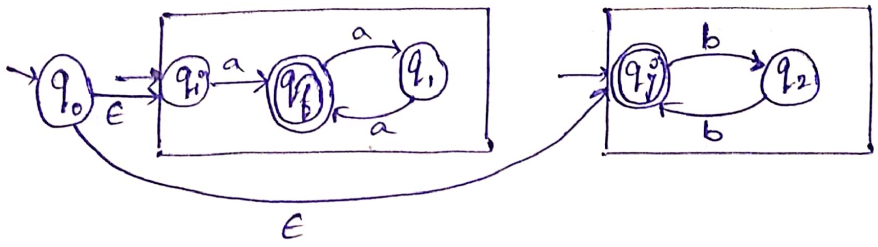
H.W. $L_i^* \in L_{FA}$

$L_i^c \in L_{FA}$

$L_i \cap L_j \in L_{FA}$

(10) $m_i = (Q_i, \Sigma, \delta_i, q_i, F_i)$ for $L_i \cup L_j \in L_{FA}$ $\Sigma = \{a\}$

$m_j = (Q_j, \Sigma', \delta_j, q_j, F_j)$ $\Sigma' = \{b\}$



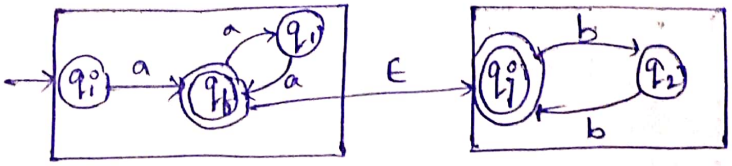
$m_{L_i \cup L_j} = (\{q_0, q_i, \ldots, q_j, \ldots\}, \Sigma_i \cup \Sigma_j, \delta, q_0, F_i \cup F_j)$ (Q_i, Q_j)

$F_U = F_i \cup F_j$

$L_{L_i \cup L_j} = \{a, aaa, bb, bbbb, aaaaa, \ldots\}$

(11) $L_i . L_j = \{abb, aaabb, abbbb, \ldots\}$, for the same alphabet.

$m_{L_i . L_j} = (\{q_i, \ldots, q_j \ldots\}, \Sigma_i \cup \Sigma_j, \delta, q_i, F_j)$ (Q_i, Q_j)



15/09/2023

{ } { }

L_{RE} = Represents sets.

U

.

*

Complementation

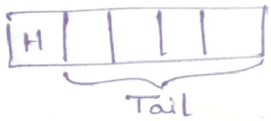
Intersection

$R_1 = \{w \mid a^*\}$ | $L_1 = \{a\}$ | $r_1 = a$

$R_2 = r_1 = (1+0).1$

$R_3 = r_3 = (ab+ba)^*$

- In an alphabet, each symbol can be a regular expression.
- $S = \{\ \}$ → set, only operation is "is a member of", i.e., $p \notin S$, cardinality
- $L = [\ \ ]$ → List ↳ elements can be repeated.
  - ↳ considers another list
  - ↳ interval (closed)
  - ↳ Ordered; ordering of positions
  - ↳ Head and Tail opp operators.
  - ↳ Length of interval.
- $O = (\ \ )$ open interval



- $O_{n,T} = (\ \ )$ → Tuples ↳ starts with 2, 3, . . . , n.
  - ↳ A tuple is one element of cross-product.

C
- Lexican
- Delimiter
- Blank

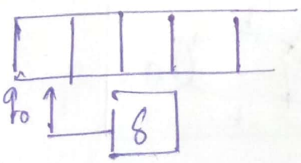
- Relation: $A R_B \subseteq A \times B$
- $M = (Q, \Sigma, \delta, q_1, F)$
- $G = (N, T, P, S)$
- $\delta(q_0, \epsilon) = q_0$

  $\delta(q_0, a) = q_f$

  $\delta(q_f, a) = q_f$

  $\delta = \{(q_0, \epsilon, q_0), (q_0, a, q_f), (q_f, a, q_f)\}$

Transition from one state to another.

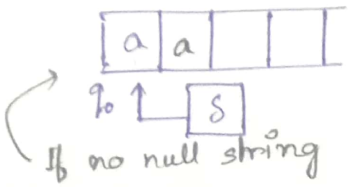


Reading tape, hence only input, no output.

If writing head, then output.

- $M = (\{q_0, q_f\}, \{a\}, \delta, q_0, \{q_f\})$

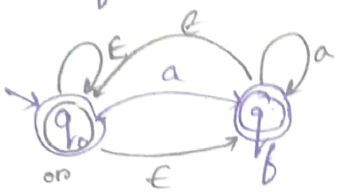
~~$\delta(q_0, a) = q_f$~~



$L = \{a\}$

$L^* = \{w \mid a^*\}$

$= \{\epsilon, a, aa, aaa, \ldots\}$



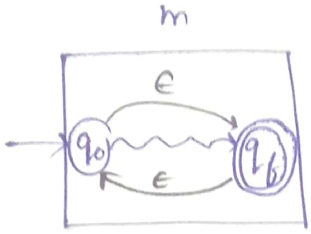
$(q_0, a) \vdash$

$\delta(q_0, a) = q_f$

$\delta(q_0, \epsilon) = q_0$

$\delta(q_f, a) = q_f$

or, $\delta(q_0, \epsilon) = q_f$



- For Kleene closure: $\delta(q_0, \epsilon) = q_f$

$\delta(q_f, \epsilon) = q_0$

- DFA and NFA difference.
- Transition Graph: $G(V, E)$, $V = \{q_0, q_1, \ldots, q_f\}$ One start state, special states as final states.

E = Symbols.

also, $= \{(q_i, q_j)$ labelled by symbols and directed from q_i to $q_j\}$

- $w = av$

$v = bx$

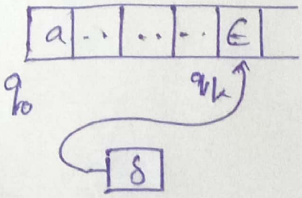
$(q_0, w) \vdash_{(i)} (q_1, v)$

$\vdash_{(ii)} (q_1, x) \ldots \vdash (q_k, \epsilon)$

i) $\delta(q_0, a) = q_1$

ii) $\delta(q_1, b) = q_1$

- $(q_0, w) \vdash^* (q_f, \epsilon)$ termination with accept



- $(q_0, w) \vdash^* (q_j, \epsilon)$ termination without accept
- $(q_0, w) \vdash^* (q_f, w')$ it is not accept configuration.